APPENDIX G: ENDNOTES

Superscript numbers in square brackets in the body text refer to the per-chapter endnotes below.

Chapter 1: What You Are Actually Being Asked to Decide

1. Identical inputs can return different outputs even with temperature set to 0, on Anthropic's own inference service and on third-party cloud providers. Anthropic, *Glossary* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/about-claude/glossary

2. The context window functions as working memory rather than storage, and accuracy and recall degrade as the token count grows, a phenomenon Anthropic names context rot. Anthropic, *Context windows* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/context-windows

3. Anthropic publishes a reliable knowledge cutoff, the date through which a model's knowledge is most extensive and reliable, separately from the training data cutoff, which is the broader range of training data used. Anthropic, *Models overview* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/about-claude/models/overview

4. Documented verification techniques include chain-of-thought verification, running the same prompt repeatedly and comparing outputs, iterative refinement, and restricting the model to supplied documents. Anthropic, *Reduce hallucinations* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/reduce-hallucinations

5. Tools are the documented mechanism for connecting Claude to external systems and APIs, so that an outside system rather than the model becomes the authority on a fact. Anthropic, *Tool use with Claude* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview

Chapter 3: Sizing, Scoping, and the Feasibility Verdict

1. Context window sizes differ by model, with a 1M-token window on some models and a 200k-token window on others, and a single request can include up to 600 images or PDF pages, or 100 on models with the smaller window. Anthropic, *Context windows* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/context-windows

2. The token counting endpoint returns the input token count for a message before it is sent, accepting the same structured input as a message creation request, including system prompts, tools, images, and PDFs. Anthropic, *Token counting* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/token-counting

3. Input and output tokens carry separate published rates on every model tier, and Anthropic directs readers to its live pricing page for the current figures. Anthropic, *Pricing* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/about-claude/pricing

4. The Message Batches API processes large volumes of requests asynchronously, reducing cost by 50 percent, with most batches finishing in less than an hour. Anthropic, *Batch processing* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/batch-processing

5. Anthropic publishes which models are active, deprecated, or retired along with retirement dates and recommended replacements, gives at least 60 days' notice before retiring a publicly released model, and fails requests sent to a model after its retirement date. Anthropic, *Model deprecations* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/about-claude/model-deprecations

Chapter 4: Augmented Call, Workflow, Agent

1. Explicit prompt chaining, described as breaking a task into sequential API calls, is recommended where intermediate outputs need inspecting or a

pipeline structure needs enforcing, with self-correction (generate a draft, review it against criteria, refine it) named as the most common chaining pattern and each step run as a separate call so it can be logged, evaluated or branched on; subagents are recommended for work that can run in parallel, needs isolated context, or forms independent workstreams; direct work is recommended for simple tasks, sequential operations, and tasks that must carry context across steps; and examples, named as few-shot or multishot prompting, are described as among the most reliable ways to steer an output's format, tone and structure. Anthropic, *Prompting best practices* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices

2. Programmatic tool calling lets Claude write code that calls defined tools from within a code execution container rather than taking a model round trip per invocation; the documented example replaces 20 separate round trips for a 20-employee budget check with a single script, and adding the capability on top of basic search tools improved performance on agentic search benchmarks by an average of 11 percent while using 24 percent fewer input tokens. Anthropic, *Programmatic tool calling* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/agents-and-tools/tool-use/programmatic-tool-calling

Chapter 5: When a Second Agent Is a Liability

1. Anthropic publishes a worked orchestration mode example in which the model scouts a task and then fans work out to parallel subagents by default; failures are isolated so one broken subagent degrades to an error string rather than ending the run, a per-request timeout bounds each call so a dropped connection degrades a single subagent, a content-addressed journal keyed on a hash of each subagent's prompt makes an interrupted run recompute only unfinished sub-tasks, a second wave of verifiers re-derives claims from source and defaults to refuted when uncertain, and subagents report through a dedicated tool so results return as structured JSON rather than prose. Anthropic, *Build an orchestration mode* (Claude Platform Docs,

accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/mid-conversation-effort-example

2. Tool use is described as a contract between the application and the model, in which the application specifies the available operations and the shape of their inputs and outputs while the model determines when and how to call them; the page names writing a regular expression to extract a decision from model output as a clear sign that the decision should have been a tool call. Anthropic, *How tool use works* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works

3. When Claude issues several tool calls in one turn, one result must be returned for each call, together in the next message and matched to its call by identifier; a call the caller chooses not to run must still be answered with a result marked as an error carrying a brief explanation. Anthropic, *Parallel tool use* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/agents-and-tools/tool-use/parallel-tool-use

4. A tool-use response can contain a server tool call whose identifier has no matching result block, meaning that call is unfinished and its result is not carried in the response; Anthropic states there is no other marker for this state and that it is detected by checking each call's identifier for a matching result block. The same page documents a paused turn returned when a server-side sampling loop reaches its iteration limit, with a documented default of 10, which the caller handles by continuing the conversation. Anthropic, *Stop reasons and fallback* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/handling-stop-reasons

5. Task budgets give Claude a token allowance for a full agentic loop, shown as a server-injected countdown the model paces itself against; Anthropic states the budget is a soft hint rather than a hard cap, that the model may exceed it when interrupting an action would be more disruptive than finishing it, and that the enforced ceiling on generation remains the request's maximum

token setting. Anthropic, *Task budgets* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/task-budgets

Chapter 6: Reference Architectures and Retrieval Against Live State

1. Anthropic documents a citations capability that grounds responses in supplied source documents and returns the exact passages supporting each claim, so answers can be verified against their sources. Anthropic, *Citations* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/citations

2. The Files API is described as a create-once, use-many-times approach: a file is uploaded once and referenced by its file identifier in later Messages requests rather than re-uploaded with each request. Anthropic, *Files API* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/files

3. Element references handed to the model are scoped to the tab that produced them and stay valid until that tab navigates or its DOM changes materially; Anthropic states the API cannot detect a stale or unknown reference, so the executor must return an error result telling the model the reference is stale, and warns against renumbering references already handed out for a tab because doing so silently invalidates references the model still holds. Anthropic, *Browser use tool* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/agents-and-tools/tool-use/browser-use-tool

Chapter 7: Model, Context Window, Context Strategy

1. The context window is described as working memory for the model rather than storage, with the note that more context is not automatically better because accuracy and recall degrade as token count grows; everything in the request counts toward the window, including the system prompt, every message with its tool results, images and documents, the tool definitions, and

the output generated for the turn, with each response reporting consumption in its usage field; cached prompt prefixes still occupy the window, since caching changes what is paid for those tokens rather than whether they count; Claude Opus 5, Claude Opus 4.8, Claude Opus 4.7, Claude Opus 4.6, Claude Sonnet 5 and Claude Sonnet 4.6 carry a 1M-token context window with up to 128k output tokens per request, while other models including Claude Sonnet 4.5 have a 200k-token window; Claude Sonnet 5, Claude Sonnet 4.6, Claude Sonnet 4.5 and Claude Haiku 4.5 have context awareness, receiving their total token budget in the system prompt of every request and a remaining-capacity update after each tool call, while Claude Opus 4.7 and later Opus models, Claude Fable 5 and Claude Mythos 5 do not receive those injected tags; input alone exceeding the window returns a 400 invalid request error on every model, while on Claude 4.5 models and newer a request whose input plus max_tokens exceeds the window is accepted and stops with a model_context_window_exceeded stop reason if generation reaches the limit. Anthropic, *Context windows* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/context-windows

2. Claude 4.6 and later models include the full 1M-token context window at standard pricing, so a 900k-token request is billed at the same per-token rate as a 9k-token request. Anthropic, *Pricing* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/about-claude/pricing

3. Anthropic advises starting with Claude Opus 5 for complex agentic coding and enterprise work when unsure which model to use, and publishes per-million-token pricing of $10 input and $50 output for Claude Fable 5, $5 and $25 for Claude Opus 5, $2 and $10 for Claude Sonnet 5, and $1 and $5 for Claude Haiku 4.5. Anthropic, *Models overview* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/about-claude/models/overview

4. Two starting approaches are documented for model selection: a cost-first approach beginning with a faster, more cost-effective model such as Claude Haiku 4.5, which allows quick iteration and lower development costs and is often sufficient, and a capability-first approach that implements with the

strongest starting point for the task and then optimises to more efficient models later. Anthropic, *Choosing the right model* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/about-claude/models/choosing-a-model

5. Thinking has Claude work through a problem in its own words before answering, and the tokens spent reasoning are billed as output tokens even when the thinking text is not returned, counting toward max_tokens alongside the response text. Anthropic, *Thinking* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/thinking

6. On adaptive models no thinking token budget is set; two controls bound cost, with max_tokens a hard cap on total output for the request that Claude never generates past and effort acting as soft guidance on how much of that output goes to thinking, and in a tool-use loop each request has its own max_tokens so it does not bound the whole turn's spend; the per-level thinking behaviour runs from always thinking with no constraint on depth at the top, through moderate thinking that may skip simple queries at medium, to minimised thinking that skips simple tasks where speed matters most at low. Anthropic, *Steering thinking* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/thinking-steering-and-cost

7. The effort parameter controls how many tokens Claude spends when responding, trading response thoroughness against token efficiency, with levels max, xhigh, high, medium and low, where high is the default and equivalent to not setting the parameter. Anthropic, *Effort* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/effort

8. Task budgets declare how many tokens Claude has for a full agentic loop, including thinking, tool calls, tool results and output, with the model shown a running countdown that it uses to prioritise work and finish gracefully as the budget is consumed; they complement the effort parameter, which controls how thoroughly Claude reasons about each step, by capping the total work

across the loop. Anthropic, *Task budgets* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/task-budgets

9. Manual extended thinking sets a fixed thinking token budget per request with budget_tokens; that budget is documented as deprecated, with a migration to adaptive thinking controlled by effort, and on models that support only extended thinking, including Claude Sonnet 4.5, Claude Opus 4.5 and Claude Haiku 4.5, requesting adaptive thinking returns a 400 error. Anthropic, *Extended thinking* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/extended-thinking

10. Compaction extends effective context length by automatically summarising older context when a conversation approaches the window limit, and also keeps the active context small because response quality degrades as a conversation grows; on later requests the API drops all content blocks prior to the compaction block; the trigger defaults to 150,000 input tokens and its value must be at least 50,000; the default summarisation prompt varies by model and instructs Claude to record the state, next steps and learnings needed to continue the task in a future context window; and custom instructions supplied through the instructions parameter replace the default prompt completely rather than supplementing it. Anthropic, *Compaction* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/compaction

11. Context editing selectively clears content from conversation history as it grows, described as active curation on the grounds that context is a finite resource with diminishing returns and irrelevant content degrades model focus; tool result clearing invalidates cached prompt prefixes when content is cleared, and a clear_at_least parameter exists so that a minimum number of tokens is cleared each time and the cache invalidation is worthwhile. Anthropic, *Context editing* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/context-editing

Chapter 8: Prompts as Architecture

1. Prompting guidance for current Claude models: be specific and explicit about the desired output and request above-and-beyond behaviour rather than relying on inference; the golden rule that a prompt confusing to a colleague with minimal context will also confuse Claude; examples described as among the most dependable methods for steering how output is formatted, how it sounds and how it is arranged, wrapped in example tags so Claude can distinguish them from instructions; the note that Claude Opus 4.5 and Claude Opus 4.6 are more responsive to the system prompt, so prompts designed to reduce tool undertriggering may now overtrigger and aggressive language such as “CRITICAL: You MUST use this tool when…” should be dialled back to ordinary phrasing; and that prefilled responses on the last assistant turn are no longer supported starting with Claude 4.6 models and Claude Mythos Preview, with such requests returning a 400 error. Anthropic, *Prompting best practices* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices

2. Claude Sonnet 5 performs well out of the box on existing Claude Sonnet 4.6 prompts, and the page covers the behaviours that most often require tuning, including response length calibrated to task complexity rather than a fixed verbosity. Anthropic, *Prompting Claude Sonnet 5* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/prompting-claude-sonnet-5

3. Claude Opus 5’s default user-facing responses run longer than prior Opus models’, and the effort parameter controls how much the model thinks rather than how much it says, so lowering effort does not reliably shorten the visible response and response length should be prompted for explicitly. Anthropic, *Prompting Claude Opus 5* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/prompting-claude-opus-5

4. Claude Fable 5 and Claude Mythos 5 have behavioural differences from Claude Opus 4.8 that may require prompt or scaffolding updates, and a

capability improvement at that level is itself a reason to re-evaluate which instructions, tools and guardrails are still needed. Anthropic, *Prompting Claude Fable 5* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/prompting-claude-fable-5

5. Techniques for increasing output consistency include specifying the desired output format precisely, prefilling the assistant turn to bypass Claude's preamble and enforce a structure, constraining with examples, grounding in retrieval, chaining prompts for complex tasks, and using system prompts to set a role. Anthropic, *Increase output consistency* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/increase-consistency

6. Leak-resistant prompt engineering should be used only when necessary because the added complexity can degrade performance on the rest of the task; the strategies given are separating context from queries in the system prompt, emphasising key instructions in the user turn and reemphasising them by prefilling the assistant turn, with a note that prefilling is not supported on Claude 4.6 and later models and Claude Mythos Preview, plus post-processing filters, omitting proprietary detail the task does not need, and periodic audits. Anthropic, *Reduce prompt leak* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/reduce-prompt-leak

7. The prompt engineering guide assumes success criteria and a way to evaluate against them are already established, and notes that not every success criterion or failing evaluation is best solved by prompt engineering. Anthropic, *Prompt engineering overview* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview

8. Structured outputs constrain responses to a schema through constrained decoding; string enum and const values are not guaranteed to match the schema's capitalisation, with the response completing normally, no error and no special stop reason, so enum values should be compared case-insensitively

and values differing only in capitalisation avoided; and several SDKs transform schemas with unsupported features, stripping numeric and length constraints from the schema that is sent, moving them into the description, and validating the response against the original constraint in the caller's own code. Anthropic, *Structured outputs* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/structured-outputs

9. Defences against prompt injection from untrusted third-party content are structural: deliver such content only inside tool result blocks and never in system prompts or plain user text blocks, since Claude is trained to treat instructions inside tool results with appropriate scepticism; state in the tool's description what the content is and where it came from; state in the system prompt that content returned from tools, documents or searches is untrusted data that must never override the system prompt or the user's request; JSON-encode untrusted strings so an attacker cannot break out of the payload into an instruction context; do not place your own instructions in tool results, because content there is treated as untrusted data and may be ignored or flagged as an injection attempt; and screen tool output with a lightweight model such as Claude Haiku 4.5 using structured outputs so the verdict is a parseable value the application can branch on. Anthropic, *Mitigate jailbreaks and prompt injections* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/mitigate-jailbreaks

10. Prompt caching resumes from a cached prefix: static content is placed at the beginning of the prompt, cache prefixes are created in the order tools, then system, then messages, and a change at one level invalidates that level and all subsequent ones, so modifying tool definitions invalidates the entire cache; cache writes happen only at a breakpoint, and a read walks backward from the breakpoint looking for entries prior requests wrote, checking at most 20 positions, so placing a breakpoint on a block that changes every request finds nothing, a trap automatic caching also hits when the last cacheable block is the varying one; cache hits require 100% identical prompt segments up to the breakpoint; the default lifetime is 5 minutes and is

refreshed at no additional cost each time the entry is used, with a 1-hour lifetime available at twice the base input token price; up to 4 breakpoints may be defined and the breakpoints themselves add no cost, since charging is based on what content is written and read; prompt caching has no effect on output token generation; and caches are isolated between organisations and, on the Claude API, between workspaces. Anthropic, *Prompt caching* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/prompt-caching

11. Without cache diagnostics the only signal of an unexpected cache miss is the cache read token count dropping to zero, with no indication of what changed; passing the previous response id has the API compare the two requests and report the first point of divergence among the model, the system prompt, the tools and the message history, with the diagnostics field answering whether the request changed while the usage field answers whether the cache hit; documented divergence types include a changed model, a system prompt carrying an interpolated timestamp or request id (with the fix being to make the system prompt byte-stable and move dynamic data into the first user message after the breakpoint), a tools array that was added to, reordered or serialised non-deterministically, and message history that was altered or truncated rather than appended to. Anthropic, *Cache diagnostics* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/cache-diagnostics

12. Agent Skills package instructions, scripts and resources as versioned units; for production Anthropic advises pinning a specific version, because omitting the version or setting it to latest means a version uploaded by anyone in the workspace immediately changes what deployed agents run; a new version is a complete snapshot rather than a delta, so the full file set must be uploaded each time and omitted files are not carried over; Skills render into the system prompt in a fixed order, so changing the Skills list or its order breaks the prompt cache, and with an unpinned version publishing a new version can invalidate the cached prefix if it changes the Skill's description; and where the Compliance API is enabled its activity feed records creation and deletion of Skills and Skill versions, while operations occurring while it is off are not

recorded and cannot be recovered later. Anthropic, *Using Agent Skills with the API* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/skills-guide

Chapter 9: Chunking, Indexing, and the Query You Did Not Anticipate

1. Search result content blocks let Claude attribute answers to supplied content; the page states that the text block is the minimal citable unit, that Claude cites whole blocks rather than substrings within a block, and that finer-grained citations are obtained by splitting search result content into smaller blocks. The same page states that citations are disabled by default for search results, and advises that when a search fails or returns nothing the tool should return a plain text block describing the outcome rather than raising an error, so that the model explains the empty result and the conversation continues. Anthropic, *Search results* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/search-results

2. Anthropic documents three document types for citations: plain text documents, which are automatically chunked into sentences and cited by character indices; PDF files with text content, also chunked into sentences and cited by page number; and custom content, which receives no additional chunking and is cited by block indices, and is recommended where more granular citations are wanted. Anthropic, *Citations* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/citations

3. Each page of a PDF typically uses 1,500 to 3,000 tokens depending on content density; Anthropic contrasts a text-extraction-only mode using approximately 1,000 tokens for a 3-page PDF with a full visual understanding mode that processes each page as both text and image and uses approximately 7,000 tokens for the same 3 pages. The page also states that PDF support relies on Claude's vision capabilities and is therefore subject to the same considerations as other vision tasks. Anthropic, *PDF support* (Claude Platform

Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/pdf-support

4. Anthropic states that it does not offer its own embedding model and directs readers to a third-party embeddings provider; the page's worked retrieval example embeds documents and queries with different input types, marking which side of the retrieval the text sits on. Anthropic, *Embeddings* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/embeddings

5. The tool search tool is documented as the way to keep a very large tool catalogue usable: rather than loading every definition into the context window at the outset, Claude searches the catalogue and loads only the definitions a task needs. Two variants are documented, one in which Claude constructs regular expression patterns and one in which Claude issues natural language queries against BM25, and the page notes that a custom tool search can be implemented using embeddings or semantic search. Anthropic, *Tool search tool* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-search-tool

Chapter 10: Entry Points, Routes, and the Integration Layer

1. The connector page documents per-tool configuration, including enabling and disabling individual tools with allowlist and denylist patterns, deferring a tool description so it is not sent to the model initially, and a recommendation to deny write or destructive tools when building read-only assistants or when a human confirmation step before state changes is wanted; it also notes that clear, specific tool descriptions improve selection accuracy across large tool sets, and states that the connector is not covered by zero-data-retention arrangements, with data exchanged with servers, including tool definitions and execution results, retained under Anthropic's standard retention policy. Anthropic, *MCP connector* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/agents-and-tools/mcp-connector

2. The Model Context Protocol is published as an open protocol for connecting language model applications to external data sources and tools, using JSON-RPC 2.0 messages between hosts that initiate connections, clients within those hosts, and servers that provide context and capabilities; servers offer resources, prompts, and tools, and the specification compares its role to the way the Language Server Protocol standardized language support across development tools. Model Context Protocol, *Specification* (accessed 2026-08-25), https://modelcontextprotocol.io/specification/draft

3. The specification's architecture section describes a client-host-server arrangement in which the host controls connection permissions, enforces security policies and consent requirements, and handles user authorization decisions, and states as a design principle that servers should not be able to read the whole conversation or see into other servers, with full conversation history staying with the host. Model Context Protocol, *Architecture* (accessed 2026-08-25), https://modelcontextprotocol.io/specification/draft/architecture

4. Tunnels are documented as a way to reach servers running inside a private network over an outbound-only connection, removing the need to open inbound firewall ports, expose services publicly, or allowlist Anthropic's address ranges at the origin. Anthropic, *MCP tunnels* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/agents-and-tools/mcp-tunnels/overview

5. By default any user running Claude Code can connect any MCP server they choose; Anthropic states that it reviews connectors against its listing criteria before adding them to its directory but does not security-audit or manage any MCP server. Administrators can restrict this across a documented range of patterns, from disabling servers entirely or deploying a fixed managed set users cannot add to, through a published approved catalogue, to leaving users unrestricted. Anthropic, *Control MCP server access for your organization* (Claude Code Docs, accessed 2026-08-25), https://code.claude.com/docs/en/managed-mcp

6. The authorization specification states that authorization is optional for implementations, that HTTP-transport implementations should conform to it while implementations using standard input and output should instead take credentials from the environment, and that servers must validate that access tokens were issued specifically for them as the intended audience, must only accept tokens valid for their own resources, and must not accept or transit any others. Model Context Protocol, *Authorization* (accessed 2026-08-25), https://modelcontextprotocol.io/specification/draft/basic/authorization

7. The security best practices document names token passthrough as an explicitly forbidden anti-pattern and sets out its consequences: circumvention of controls that depend on token audience, downstream logs that show requests appearing to come from a different identity, and use of the server as a proxy for exfiltration where claims are not validated. The same document covers the confused deputy problem and a scope minimization section describing the risks of broad up-front scopes, including expanded blast radius, revocation that disrupts all workflows, consent abandonment, and what it names scope inflation blindness, where a lack of metrics normalizes over-broad requests; the recommended mitigation is a minimal initial scope set with targeted incremental elevation and logged elevation events. Model Context Protocol, *Security Best Practices* (accessed 2026-08-25), https://modelcontextprotocol.io/specification/draft/basic/security_best_practices

8. Anthropic documents its Amazon route as running on AWS-managed infrastructure with zero operator access, stating that Anthropic personnel have no access to the inference infrastructure; the page also documents three authentication paths, notes that model identifiers carry a provider prefix, and lists the current computer and browser tool sets as not available on that route while earlier beta versions remain. Anthropic, *Claude in Amazon Bedrock* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/claude-in-amazon-bedrock

9. On Google Cloud the request format differs in two ways: the model is specified in the endpoint URL rather than passed in the request body, and a version field is passed in the body rather than as a header. The page also states that lifecycle terms such as deprecation and retirement are set by the partner on partner-operated platforms and can differ from the Claude API schedule. Anthropic, *Claude on Google Cloud* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/claude-on-vertex-ai

10. Microsoft Foundry offers two hosting options chosen at deployment configuration time, one running on Azure infrastructure and one on Anthropic infrastructure, differing in which models are available and which deployment types are supported; usage is billed through the Azure Marketplace, and a deployment name cannot be changed after creation. Anthropic, *Claude in Microsoft Foundry* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/claude-in-microsoft-foundry

11. Three authentication methods are documented: API keys, Workload Identity Federation for workloads that already hold a platform-issued identity, and App Attest for iOS and macOS applications distributed to end users. Keys can be created with preset or custom expirations, and the page states that expiration limits the lifetime of a leaked credential but is not a substitute for secret hygiene. Anthropic, *Authentication* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/manage-claude/authentication

12. Workspaces are documented as a way to separate projects, environments, or teams within one organization while keeping billing and administration central, with API keys, members, and resource limits assignable per workspace and members able to hold different roles in each. Anthropic, *Workspaces* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/manage-claude/workspaces

Chapter 11: The Numbers Before You Build

1. Anthropic's cost guidance reports measured results for prompt caching, token trimming, effort levels, model choice, budgets and multi-model strategies, and states that all figures are Anthropic-internal benchmark runs from July and August 2026 at the list prices of the time, directional rather than guaranteed, and that readers should measure on their own workload. Reported findings include: 25 aggregate questions over a 1,862-row public CSV answered 6 of 25 correctly at $5.01 per run when the table was pasted into the prompt as roughly 91,000 input tokens, against 25 of 25 at $0.40 when the file was uploaded and queried with code execution, with Claude Opus 5 showing the same pattern at $13.45 and $1.91; a triage job whose one-line final answer used 39 percent fewer output tokens and cost $0.49 against the original two-line format's $0.57, while a five-section memo used six times the output tokens and cost $1.40, all three inside run-to-run noise on accuracy; batch processing described as the second-largest free lever after caching for unattended work; 2 problems carrying 43 percent of the spend on a 20-problem run, with the instruction to compare models on the hardest tenth of tasks rather than the typical one because a failed task still bills its tokens and then the retry; low effort giving up 1 to 3 points for a third to a half off cost per task on research and knowledge-work benchmarks, with medium matching the default's accuracy at 70 to 85 percent of its cost, against long-horizon coding where Claude Opus 5 gave up about 2 points at medium for half the cost and about 8 points at low for a quarter; low effort taking 4.5 minutes per problem against 7.9 at the default on one benchmark, and 7.9, 9.1 and 11.4 hours per episode at low, medium and the default on a corpus benchmark; and a re-run policy solving about 93 percent of tasks at about $0.70 each against 91.7 percent at $1.39 for running everything at the default, subject to having a failure signal and to each first-pass failure costing two runs of wall-clock time. Anthropic, *Optimizing for cost and intelligence* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/about-claude/models/optimizing-for-cost-and-intelligence

2. The Message Batches API processes large volumes of requests asynchronously with all usage charged at 50 percent of standard prices, most batches finishing in under an hour, requests that reach the 24-hour expiration going unbilled, and results available for 29 days after the batch is created. Anthropic, *Batch processing* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/batch-processing

3. Cache reads are billed at 0.1 times the base input rate, a 5-minute cache write at 1.25 times and a 1-hour cache write at 2 times, so caching pays off after one read for the 5-minute duration and after two for the 1-hour duration; pinning inference to the United States on Claude 4.6 and later models applies a 1.1x multiplier to every pricing category including cache writes and reads; and the tool use system prompt adds 286 tokens on Claude Opus 5 for automatic or no tool choice and 406 tokens when a specific tool or any tool is required, while the computer toolset adds about 4,500 input tokens to a request and the browser toolset about 6,600. Anthropic, *Pricing* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/about-claude/pricing

4. Latency is defined as the delay between submitting a prompt and receiving the generated output, and time to first token is defined separately as the time taken to generate the first token of the output after a prompt is received. Anthropic, *Glossary* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/about-claude/glossary

5. Three approaches to reducing latency are documented: selecting a faster model, with Claude Haiku 4.5 named as offering the fastest response times for speed-critical applications; minimising the token count in both prompt and expected output, including a hard cap on generated length; and streaming, which lets the model start returning its response before generation is complete and is described as improving the perceived responsiveness of an application. Anthropic, *Reducing latency* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/reduce-latency

6. Fast mode delivers up to 2.5x higher output tokens per second from Claude Opus 5 and Claude Opus 4.8 at premium pricing across the full context window, running the same model with a faster inference configuration and no change to capability, with a dedicated rate limit that returns a 429 carrying a retry-after header when exceeded; requesting it on an unsupported model returns an error except on Claude Opus 4.6, which silently runs at standard speed while reporting the speed accurately in the response usage object; and the Anthropic software development kits retry rate-limited requests up to 2 times by default, with the note that setting the retry count to zero also disables retries for other transient errors including overloaded and internal server errors. Anthropic, *Fast mode* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/fast-mode

7. A refusal is returned as a successful HTTP 200 response rather than an error, and server-side fallback retries the request on another model within a single API call, either by default routing based on the refusal category or against a list of up to three named models; only a safety classifier decline triggers the fallback, and a rate limit, overload or server error on the requested model is returned as-is. Anthropic, *Refusals and fallback* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/refusals-and-fallback

8. Prompt caches are held per model, so retrying a refused request on a different model would otherwise require writing the already-cached conversation prefix into the second model's cache at the higher write rate; a credit token minted on the refusal and echoed on the retry causes the retry to be billed as though the conversation had been on the new model throughout. Anthropic, *Fallback credit* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/fallback-credit

9. Every response carries a field reporting why generation stopped, with distinct values for a natural finish, reaching the output token limit, hitting a custom stop sequence, calling a tool, a paused turn, a refusal, and exceeding the context window; the documentation advises checking it in response-handling

logic and distinguishing these values from actual errors. Anthropic, *Stop reasons and fallback* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/handling-stop-reasons

Chapter 12: Observability at Scale

1. Streaming responses use server-sent events, and the documentation notes that the API may send errors within the event stream, giving an overloaded error as an example and stating that it would normally correspond to an HTTP 529 status in a non-streaming context. Anthropic, *Streaming messages* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/streaming

2. The monitoring reference documents a separate refusal event alongside the API error event, explaining that refusals arrive on a successful response stream rather than as an HTTP error so the error event does not fire for them; it states that distributed tracing is off by default and requires two environment variables to enable; it describes a span hierarchy in which each user prompt opens a root interaction span with API calls, tool calls and hook executions recorded as children and a subagent's spans nested under the parent tool span; and it documents environment variables for metrics cardinality control, noting that lower cardinality generally means better performance and lower storage costs but less granular data for analysis. Anthropic, *Monitoring* (Claude Code Docs, accessed 2026-08-25), https://code.claude.com/docs/en/monitoring-usage

3. Prompt caching requires the beginning of a prompt to be byte-for-byte identical to a recent request, and without the cache diagnostics feature the only available signal of a miss is the cache read token count dropping to zero with no indication of what changed; the feature compares a request against a previous response id and reports the first point of divergence among the model, the system prompt, the tools and the message history, reporting only the earliest divergence so that later ones may be hidden behind it. Anthropic, *Cache diagnostics* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/cache-diagnostics

4. Telemetry from the Agent SDK is off until an enabling environment variable is set and at least one exporter is chosen; the SDK produces no telemetry of its own and instead passes configuration through to the instrumented command-line process, which records spans around each model request and tool execution and exports directly to the configured collector. Anthropic, *Observability with OpenTelemetry* (Claude Code Docs, accessed 2026-08-25), https://code.claude.com/docs/en/agent-sdk/observability

5. The organisation usage report endpoint returns token consumption grouped and filtered by model, workspace, API key, service tier, context window, inference region and speed, at bucket widths of one minute, one hour or one day, with the one-minute width documented for real-time monitoring, the one-hour width for daily patterns and the one-day width for weekly and monthly reports; a separate cost report endpoint returns service-level cost breakdowns in United States dollars. Anthropic, *Usage and Cost API* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/manage-claude/usage-cost-api

Chapter 13: Evals as Acceptance Criteria

1. Building an application on a large language model starts with defining success criteria and then designing evaluations to measure performance against them, a cycle described as central to prompt engineering; good criteria are specific, measurable, achievable and relevant, with achievability based on industry benchmarks, prior experiments, research or expert knowledge rather than targets beyond current frontier model capability, and a well-formed safety criterion is given as fewer than 0.1 percent of outputs across 10,000 trials flagged for toxicity by the content filter. On grading, the guidance is to choose the fastest, most reliable and most scalable method available: code-based grading, illustrated with an equality check against a golden answer and a key-phrase presence check, is fastest and most reliable and extremely scalable but lacks nuance for judgments requiring less rule-based rigidity; large-language-model-based grading is fast, flexible, scalable and suitable for complex judgment, to be tested for reliability before scaling; and human grading is the most flexible and highest quality but slow and expensive, to be

avoided if possible. Anthropic, *Define success criteria and build evaluations* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/test-and-evaluate/develop-tests

2. When an outcome is defined, the harness provisions a grader to evaluate the artifact against a rubric, and the grader uses a separate context window so that it is not influenced by the main agent implementation choices. The rubric is a markdown document describing per-criterion scoring, and it is required. Anthropic, *Define outcomes* (Claude Platform Docs, accessed 2026-08-28), https://platform.claude.com/docs/en/managed-agents/define-outcomes

3. The Message Batches API processes large volumes of requests asynchronously at 50 percent of standard pricing with increased throughput, with most batches finishing in under an hour and each request processed independently so that different request types can be mixed in one batch; results are returned in a format where order is not guaranteed and a caller-supplied custom identifier is used to match results to requests, with the documentation's own example output listing a second request ahead of a first; each request ends in one of four result types, succeeded, errored, canceled or expired, where expired means the batch reached its 24-hour expiration before the request was sent to the model; and results remain available for 29 days after batch creation. Anthropic, *Batch processing* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/batch-processing

4. Structured outputs constrain responses to a supplied schema through constrained decoding, and the documentation states that while schema compliance is guaranteed in most cases there are scenarios where the output may not match the schema, naming a refusal, which returns a stop reason of refusal, and reaching the maximum token limit, which returns a stop reason of max_tokens with the structured output cut off before completion. Anthropic, *Structured outputs* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/structured-outputs

5. The API handles refusals in three different ways: a stop reason of refusal returned when streaming classifiers intervene on potential policy violations,

400 error codes when input or copyright validation fails, and standard text responses when the model itself refuses; the page instructs that a refusal be handled as its own outcome rather than as an errored result, and notes that monitoring built only on error rates will not surface it. Anthropic, *Handle streaming refusals* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/handle-streaming-refusals

Chapter 14: A/B, Shadow, and Not Overclaiming

1. Among the documented techniques for reducing hallucinations is best-of-N verification, which runs Claude through the same prompt multiple times and compares the outputs on the basis that inconsistencies across those outputs can indicate hallucination; the page also lists chain-of-thought verification, iterative refinement, and restricting the model to information in the provided documents rather than its general knowledge. Anthropic, *Reduce hallucinations* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/reduce-hallucinations

2. Each Claude model ID identifies a pinned version whose underlying model remains constant for the lifetime of that ID, a guarantee that covers model IDs and not the convenience aliases the Claude API accepts for some earlier models, where such an alias resolves to the most recent dated snapshot for its minor version. From the 4.6 generation onward, IDs use a dateless format, and the documentation corrects the common misconception that these behave as evergreen pointers to the latest or best-performing version: the dateless ID is the canonical ID for that release, maps to a single fixed snapshot, is never updated in place, and an updated version ships under a new model ID. Model weights are fixed for a given ID, but the serving infrastructure around the model, including the request router, safety classifiers and sampling logic, can change over time, and infrastructure updates occasionally produce minor differences in observable behaviour even when the model ID and weights have not changed; where unexpected behavioural differences appear on a previously stable model ID, an

infrastructure update is named as the most likely cause. Anthropic, *Model IDs and versioning* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/about-claude/models/model-ids-and-versions

3. Anthropic publishes which models are active, deprecated or retired together with retirement dates and recommended replacements, and advises thorough testing of applications against replacement models well before the retirement date in order to measure the replacement's performance on the relevant tasks. Anthropic, *Model deprecations* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/about-claude/model-deprecations

4. Almost any Messages API request can be included in a batch, but a small number of parameters are unsupported and including any of them returns a validation error: streaming, because batch results are returned as a single file rather than a stream; the fast-mode speed parameter, because it tunes synchronous latency which does not apply to asynchronous processing; the stateful thread parameters, because batch requests are not stateful; and the cache and context routing hints, because they apply only to synchronous request scheduling. Anthropic, *Batch processing* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/batch-processing

5. Techniques for minimising hallucination include allowing Claude to say that it does not know, restricting it to information contained in the provided documents rather than its general knowledge, requiring it to identify the supporting passages before answering, and verifying claims afterwards, with human review used where accuracy is critical. Anthropic, *Reduce hallucinations* (Claude Platform Docs, accessed 2026-08-28), https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/reduce-hallucinations

Chapter 15: The Safety Stack, and Where the Guardrails Sit

1. Documented mitigations against a deliberately adversarial user include pre-screening input with a lightweight model such as Claude Haiku 4.5, constrained by structured outputs to a simple classification, before it reaches the main conversation; filtering input for known injection patterns, including using a model given known jailbreaking language as examples to build a generalized validation screen; writing system prompts that emphasise ethical and legal boundaries and explicitly tell Claude how to refuse; and responding to repeat offenders by adjusting responses, throttling or banning users who repeatedly attempt to circumvent the application's guardrails. Against instructions embedded in content Claude reads on a user's behalf, the page directs applying the principle of least privilege so a successful injection does minimal damage, withholding secrets Claude does not need, running tools in sandboxed environments and scoping permissions narrowly; screening tool outputs by passing each tool's raw output to a small classifier call with Claude Haiku 4.5 and returning the content only if the screen reports no injection attempt, using structured outputs so the verdict is a parseable value the application can branch on; red-teaming the workflow before deployment with documents, emails and tool outputs that deliberately contain injection attempts; and regularly analysing outputs for signs of successful injection to refine prompts, validation and filtering. Anthropic, *Mitigate jailbreaks and prompt injections* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/mitigate-jailbreaks

2. Permission modes govern which actions run without asking. The classifier that reviews actions sees user messages, non-read-only tool calls and project instruction file content, while tool results are stripped so hostile content in a file or web page cannot manipulate it directly, and a separate server-side probe scans incoming tool results and flags suspicious content before Claude reads it. Boundaries stated in conversation are treated as a block signal and stay in force until lifted, but are not stored as rules: the classifier re-reads

them from the transcript on each check, so a boundary can be lost if context compaction removes the message that stated it, and the page recommends adding a deny rule for a hard guarantee. Deny rules block in every mode including bypassPermissions, allow rules have no effect in that mode, and a set of actions is auto-approved by no mode at all, including removals targeting critical paths. When the classifier produces no verdict, because a separate safety check refused its own request or its response did not parse, the action is denied; if the classifier blocks 3 actions in a row or 20 in total the mode pauses and Claude Code resumes prompting, and those thresholds are not configurable; in a non-interactive run with no prompt to fall back to, the action does not run and the work continues; and when auto mode is selected but unavailable to the session, the session starts in Manual instead. Anthropic, *Choose a permission mode* (Claude Code Docs, accessed 2026-08-25), https://code.claude.com/docs/en/permission-modes

3. Permission modes decide whether a tool call runs and whether the user is prompted first, while isolation restricts what a command can access once it runs; the action-reviewing classifier is described as a per-action control rather than an isolation boundary. The built-in sandboxed Bash tool restricts only Bash commands, leaving built-in file tools, MCP servers and hooks running directly on the host, whereas every other approach on the page places the whole Claude Code process inside the isolation boundary. Anthropic, *Choose a sandbox environment* (Claude Code Docs, accessed 2026-08-25), https://code.claude.com/docs/en/sandbox-environments

4. A classifier refusal returns a stop details object naming the policy category that triggered it plus a human-readable explanation whose text is not stable and should be displayed rather than parsed; the category field is null where a decline belongs to no named policy area, and that null is described as a normal and permanent value rather than a placeholder. The documented categories include cyber, described as covering requests that could enable cyber harm such as malware or exploit development, with the note that benign cybersecurity work can also trigger it, and bio, covering requests that could enable biological harm, with the note that beneficial life sciences work can also trigger it. A refusal can arrive before any output or mid-stream after

partial output, and partial output is to be treated as incomplete and discarded. Anthropic, *Refusals and fallback* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/refusals-and-fallback

5. On receiving a refusal stop reason, the conversation context must be reset before continuing, either by removing or rephrasing the turn that triggered the refusal or by clearing the conversation history; attempting to continue without resetting results in continued refusals. Anthropic, *Handle streaming refusals* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/handle-streaming-refusals

6. Anthropic offers Claude Enterprise organisations automated security scanning for custom Skills, in beta, checking for signs of malicious behaviour such as hidden code execution, sending data to an outside service, or instructions that tamper with Claude's safeguards; a Skill that fails the scan or whose scan has not finished is blocked from use, one that passes with a warning stays usable behind a caution notice, and the page states that scanning complements but does not replace the review checklist. Skills uploaded through the API are not scanned, nor are Skills that were already in the organisation when scanning was turned on, nor organisations with certain data handling configurations including customer-managed encryption keys, zero data retention, and HIPAA readiness. The same page states that Skill authors should not be their own reviewers. Anthropic, *Skills for enterprise* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/agents-and-tools/agent-skills/enterprise

Chapter 16: Human Review by Stakes, Not by Volume

1. Permission modes set which actions run in a session without asking first, each mode making a different trade between convenience and oversight: the mode labelled Manual, with the config value default, runs reads only without

a prompt; acceptEdits also runs file edits and common filesystem commands such as mkdir, touch, mv and cp; auto runs everything with background safety checks; dontAsk runs only pre-approved tools; and bypassPermissions runs everything. A set of actions is auto-approved by no mode at all, including bypassPermissions, among them tools matched by an explicit ask rule, connector tools an organization has set to ask, and tools that require user interaction such as the built-in AskUserQuestion tool and MCP tools marked as requiring it. In plan mode Claude researches and proposes changes without editing source, and edits stay blocked until the plan is approved; at the approval prompt the options are to approve and proceed in auto mode (or auto-accept edits where auto mode is unavailable), to approve and manually approve each edit, or to keep planning and say what to change, and Ctrl+G opens the proposed plan in a text editor for direct editing before Claude proceeds. Anthropic, *Choose a permission mode* (Claude Code Docs, accessed 2026-08-25), https://code.claude.com/docs/en/permission-modes

2. A callback passed in the query options fires whenever Claude wants to use a tool that nothing earlier in the permission flow has approved, receiving the tool name and input, and execution pauses until the callback returns; the callback can stay pending indefinitely, and where a user may take longer than the process can stay running, a deferred response lets the process exit and resume later from the persisted session. Documented responses include approving the action as requested, approving with a modified input (Claude sees the result but is not told the input changed), approving and echoing back a suggested permission rule so matching calls skip the prompt next time, with a suggestion carrying the local-settings destination written to a settings file so future sessions skip it, rejecting with a message Claude sees and may act on, suggesting an alternative, and redirecting entirely with new instructions. A permission-request hook can send external notifications to Slack, email or push when Claude is waiting for approval. Each call of the clarifying-question tool supports 1 to 4 questions with 2 to 4 options each. Anthropic, *Handle approvals and user input* (Claude Code Docs, accessed 2026-08-25), https://code.claude.com/docs/en/agent-sdk/user-input

3. Claude can return detailed citations when answering questions about supplied documents, giving the exact passages that support each claim so answers can be verified and sources surfaced; citations must be enabled on all or none of the documents in a request, and the field carrying the quoted passage is provided for convenience and counts toward neither output tokens nor input tokens. Anthropic, *Citations* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/citations

4. Structured outputs constrain Claude's responses to a specified schema through constrained decoding, so downstream processing receives valid, parseable output rather than possibly malformed JSON. Anthropic, *Structured outputs* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/structured-outputs

5. Every Messages API response includes a stop reason field explaining why generation ended, checked to decide whether to use the response as it stands, continue the conversation, retry, or fall back to another model; the tool use value indicates that Claude is calling a tool and expects the caller to run it. Anthropic, *Stop reasons and fallback* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/build-with-claude/handling-stop-reasons

Chapter 17: Proving It to Someone Who Was Not There

1. A production Compliance API integration chooses how it consumes the Activity Feed, how its output correlates with a SIEM, and where long-term copies live. The Activity Feed is to be treated as at-least-once, with deduplication on the activity id, because a retry after a partial failure can re-deliver activities already stored; even with cleanly tiled windows, an activity that indexes after its window has closed never appears in a later window, so the page directs overlapping each new window by a few minutes or running a periodic reconciliation pass over an older window. The list endpoints return neither a total_count field nor a checksum, so to attest that an export run is complete the page directs logging the window bounds or cursor, the last_id,

and the request-id of the final page; it also states that activity volume is not a completeness check. Calls to the Compliance API itself emit compliance_api_accessed activities, which can be ingested alongside other activity types so that a SIEM records who queried compliance data and when, attributed through actor.api_key_id. For chain of custody, exported records are to be stored with provenance metadata: source endpoint, query parameters, run timestamp, and a content hash of each record. The same page sets out five retention horizons and who controls each: Activity Feed records are retained for 6 years by Anthropic; chat, file and project content follows the organization's own retention policy; local session transcripts are retained for 6 years by default or for an organization's custom conversation retention period where a finite one is set; remote session transcripts are retained for 6 years; and content hard-deleted through the Compliance API is not retained, with deletion described as immediate and permanent and controlled by the caller of the delete endpoint. Anthropic, *Design your compliance integration* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/manage-claude/compliance-integration-patterns

2. Data residency is governed by two independent settings: the inference_geo API parameter, which controls where model inference runs for a request and takes the values global (the default, allowing inference in any available geography) or us (US-based infrastructure only), and the workspace geography, which is set when a workspace is created and cannot be changed afterwards. Workspace settings also support allowed_inference_geos, which restricts which geographies a workspace may use and returns an error where a request specifies one outside the list, and default_inference_geo, which sets the fallback when the parameter is omitted. The response usage object includes an inference_geo field indicating where inference ran. US inference is priced at 1.1 times the standard rate across all token pricing categories, including input tokens, output tokens, cache writes and cache reads. Anthropic, *Data residency* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/manage-claude/data-residency

3. The Usage and Cost Admin API supports grouping and filtering usage by the inference_geo dimension, described as useful for verifying geographic routing across an organization; the valid filter values are global, us, and not_available. Anthropic, *Usage and Cost API* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/manage-claude/usage-cost-api

4. Anthropic offers two data handling arrangements for the Claude API. Under zero data retention, customer prompts and responses are not stored at rest after the API response is returned; the API does not block features that are ineligible for the arrangement, and using one is described as a choice to step outside the arrangement for that specific data, with the feature's own documented retention policy applying. HIPAA readiness, which requires both a countersigned Business Associate Agreement and an organization with the arrangement enabled, applies a broader set of privacy and security safeguards across the data lifecycle, and the page states that an organization handling protected health information should use HIPAA readiness and does not also need zero data retention. Under HIPAA readiness the API enforces the feature restrictions automatically, returning a 400 error listing the non-eligible features detected in the request. Enablement is bound to the exact BAA version downloaded, and once HIPAA readiness is enabled for an organization the configuration is permanent and cannot be disabled by an administrator. When structured outputs or tools with strict: true are used, JSON schemas are compiled into grammars cached separately from message content, and those cached schemas do not receive the same protected-health-information safeguards as prompts and responses, so PHI must not appear in schema property names, enum values, const values, or pattern regular expressions. Anthropic, *API and data retention* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/manage-claude/api-and-data-retention

5. Where a customer has executed a Business Associate Agreement with Anthropic and has Zero Data Retention enabled for the relevant organization, that agreement extends to the customer's API traffic through Claude Code. Anthropic, *Legal and compliance* (Claude Code Docs, accessed 2026-08-25), https://code.claude.com/docs/en/legal-and-compliance

6. Access Transparency creates a record of human access to an organization's data by Anthropic personnel, delivered as anthropic_access activities on the existing Compliance API Activity Feed under the organization's existing Compliance Access Key. Each event carries the Anthropic team that performed the access and a reason code, while the actor object is always an Anthropic actor with a null email address, so individual employee identity is not disclosed. The time of access and the time the activity became visible in the feed are separate fields, and the access time may be earlier than the time the activity appears. Preservation of content beyond the standard retention window is itself logged as a customer-visible event carrying the same fields. Anthropic, *Access Transparency* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/manage-claude/access-transparency

Chapter 19: Feedback Loops and What You Owe When One Breaks

1. Webhook events are delivered only to endpoints subscribed to that event type at the moment the event is emitted, an event emitted while no endpoint is subscribed is never delivered and subscribing later does not backfill it; each endpoint and event receives up to three delivery attempts with jittered exponential backoff between 5 and 120 seconds, after which the event is dropped rather than queued, with no signal that it was lost; an endpoint is automatically disabled with a machine-readable reason after a redirect response, an invalid address, or sustained delivery failures, where the trigger for the last is the uninterrupted duration of failure rather than a delivery count and a single successful response resets the window; events emitted while an endpoint is disabled are not replayed on re-enabling; and ordering is not guaranteed, so state should be driven from the fetched resource, with reconciliation by listing or fetching through the API where every transition must be observed. Anthropic, *Subscribe to webhooks* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/managed-agents/webhooks

2. The Spend Limits API returns each member's effective spend limit together with a field identifying the scope level the limit was resolved from and the

member's period-to-date spend; spend limit increase requests are raised by members rather than through the API and are resolved through explicit approve and deny endpoints, with both outcomes terminal and at most one request pending per member; a denied request causes the member's request control to be hidden for 30 days from resolution while an administrator may still raise the limit directly at any time; Anthropic emails the member on approval or denial by default, with a suppression parameter documented for cases where the organisation notifies the member through its own system; and period-to-date spend is documented as informational rather than transactional and may read as zero when the spend reading is temporarily unavailable. Anthropic, *Spend Limits API* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/manage-claude/spend-limits-api

3. Impacted customers are always notified by email and in the documentation, Anthropic notifies customers with active deployments on models that have upcoming retirements, and customers can access an audit of their own API usage in order to locate any application still calling a deprecated model. Anthropic, *Model deprecations* (Claude Platform Docs, accessed 2026-08-25), https://platform.claude.com/docs/en/about-claude/model-deprecations

Chapter 21: Team Enablement and the Debugging Loop

1. Installing Claude Code creates no settings file; settings resolve through a precedence stack with managed settings highest, then command line arguments, project local settings, shared project settings and user settings; list keys such as the permission allow list merge across files rather than overriding; and a committed project file's allow rules, additional directories, declared marketplaces and most environment values apply only after each teammate trusts the folder, while deny and ask rules apply immediately. Anthropic, *Claude Code settings* (Claude Code Docs, accessed 2026-08-25), https://code.claude.com/docs/en/settings

2. Claude Code checks four managed settings sources in priority order and applies the first that delivers at least one policy key; array settings such as the permission allow and deny lists merge entries from all sources, so developers can extend managed lists but not remove from them. Anthropic, *Set up Claude Code for your organization* (Claude Code Docs, accessed 2026-08-25), https://code.claude.com/docs/en/admin-setup

3. Memory files are loaded at the start of every conversation and treated as context rather than as enforced configuration, and the documentation directs readers to a pre-tool-use hook to block an action regardless of what Claude decides. Anthropic, *How Claude remembers your project* (Claude Code Docs, accessed 2026-08-25), https://code.claude.com/docs/en/memory

4. Hooks are user-defined shell commands run at fixed points in the lifecycle, which the documentation describes as giving deterministic control so that certain actions always happen rather than relying on the model to choose to run them. Anthropic, *Automate actions with hooks* (Claude Code Docs, accessed 2026-08-25), https://code.claude.com/docs/en/hooks-guide

5. A skill's body loads only when it is used, so long reference material costs almost nothing until it is needed; skills are distributed by committing them to a repository, by shipping them inside a plugin, or by deploying them organisation-wide through managed settings. Anthropic, *Extend Claude with skills* (Claude Code Docs, accessed 2026-08-25), https://code.claude.com/docs/en/skills

6. By default a plugin dependency tracks the latest available version, so an upstream release can change the dependency under a plugin without warning; the page works through two internal teams where a tagged release moves every engineer's copy and breaks the dependent plugin, and offers a semantic version range as the constraint that holds a dependency at a tested version until the dependent team chooses to move. Anthropic, *Constrain plugin dependency versions* (Claude Code Docs, accessed 2026-08-25), https://code.claude.com/docs/en/plugin-dependencies

7. Claude Code checks for marketplace and plugin updates in the background after a session starts, with a random delay of up to ten minutes, and the running session keeps the versions it loaded at launch until the plugins are reloaded or the next session begins. Anthropic, *Discover and install prebuilt plugins through marketplaces* (Claude Code Docs, accessed 2026-08-25), https://code.claude.com/docs/en/discover-plugins

8. When a review runs, multiple agents analyse the diff and the surrounding code in parallel, a verification step checks candidate findings against actual code behaviour to filter out false positives, and the results are deduplicated, ranked by severity and posted as inline comments on the specific lines where issues were found, with a summary in the review body. Anthropic, *Code Review* (Claude Code Docs, accessed 2026-08-28), https://code.claude.com/docs/en/code-review

9. Lines of code accepted counts the lines Claude Code wrote that users accepted in their sessions, excluding rejected suggestions and not tracking subsequent deletions; the contribution metrics are described as deliberately conservative and an underestimate, counting only cases of high confidence in the tool's involvement, and code that developers substantially rewrote, defined as more than 20 percent different, is not attributed. Anthropic, *Track team usage with analytics* (Claude Code Docs, accessed 2026-08-25), https://code.claude.com/docs/en/analytics

10. The configuration debugging page documents commands for finding out what actually loaded: a context breakdown listing memory files, skills and the source each custom subagent loaded from; a status view naming the active settings sources including which managed source applies; a hooks listing; a server listing showing connection status and per-project approval; a safe mode that launches with customisations disabled; and a debug flag for a server that connects but returns no tools, whose error output is read from the session's debug log. It also names two silent failures, a misspelled tool name in a hook matcher that matches nothing, and a project-scoped server that stays disabled because its one-time approval prompt was dismissed.

Anthropic, *Debug your configuration* (Claude Code Docs, accessed 2026-08-25), https://code.claude.com/docs/en/debug-your-config

11. A non-interactive run loads the same context an interactive session would unless bare mode is used, which skips auto-discovery of hooks, skills, custom commands, subagents, plugins, servers and memory files; the documentation recommends it for continuous integration and scripts that need the same result on every machine, noting that a hook in a teammate's home directory or a server in the project's configuration will not run because bare mode never reads them. Without it, a non-interactive session runs the hooks in a project's settings file and connects the servers it declares even in a folder that has never been trusted, showing no workspace trust dialog and no per-server approval prompt. Anthropic, *Run Claude Code programmatically* (Claude Code Docs, accessed 2026-08-25), https://code.claude.com/docs/en/headless